

Trabajo Práctico Integrador: Estructuras de Datos, Contrato Equals/HashCode y Complejidad Algorítmica

Asignatura: Desarrollo de Software

Modalidad: en grupo

Criterio de Evaluación y Política de IA: Se permite el uso de herramientas de Inteligencia Artificial (LLMs). Sin embargo, el práctico está diseñado para evaluar el **razonamiento crítico, el análisis de estados de memoria y la justificación técnica**. Las respuestas genéricas generadas por IA sin prueba de trazabilidad manual o sin responder a los escenarios de borde específicos serán penalizadas.

Criterios Generales de Entrega

- Entregar un informe en formato PDF publicado en un repositorio de GitHub.
- Incluir fragmentos de código fuente en Java y los diagramas/trazas explicativas solicitadas.

Consigna 1: Diagnóstico de Bug Silencioso y Análisis de Memoria

Un equipo de desarrollo junior implementó el siguiente sistema para gestionar el acceso de empleados a un edificio mediante tarjetas de identificación mutable. El código no lanza excepciones en tiempo de ejecución, pero presenta comportamientos anómalos en producción.

```
import java.util.*;

class TarjetaAcceso {
    private String codigo;
    private int nivelAcceso;

    public TarjetaAcceso(String codigo, int nivelAcceso) {
        this.codigo = codigo;
        this.nivelAcceso = nivelAcceso;
    }

    public void setNivelAcceso(int nivelAcceso) {
        this.nivelAcceso = nivelAcceso;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        TarjetaAcceso tarjeta = (TarjetaAcceso) o;
        return nivelAcceso == tarjeta.nivelAcceso && Objects.equals(codigo,
tarjeta.codigo);
    }

    @Override
    public int hashCode() {
        return Objects.hash(codigo, nivelAcceso);
    }
}

public class ControlAcceso {
```

```

public static void main(String[] args) {
    Set<TarjetaAcceso> tarjetasValidas = new HashSet<>();[cite: 3]
    TarjetaAcceso t1 = new TarjetaAcceso("ACC-101", 2);

    tarjetasValidas.add(t1);

    // Modificación posterior a la inserción
    t1.setNivelAcceso(5);

    System.out.println("¿Tarjeta válida en sistema?: " +
tarjetasValidas.contains(t1));
    System.out.println("Cantidad de tarjetas registradas: " +
tarjetasValidas.size());
}
}

```

Responder y Justificar:

1. **Traza del Estado Interno:** Explique paso a paso qué ocurre dentro de la estructura HashSet:
 - Al ejecutar tarjetasValidas.add(t1).
 - Al ejecutar t1.setNivelAcceso(5).
 - Al ejecutar tarjetasValidas.contains(t1).
2. **Explicación del Cajón (Bucket):** ¿En qué número/ubicación de cajón quedó guardado el objeto y en cuál lo busca Java al hacer contains()?
3. **Fuga de Memoria (Memory Leak):** ¿Por qué este patrón genera una fuga de memoria si se ejecuta repetidamente en un servidor que opera 24/7?
4. **Refactorización Obligatoria:** Aplique dos soluciones arquitectónicas distintas para corregir el problema (una basada en inmutabilidad y otra ajustando el criterio de la entidad).

Consigna 2: Selección y Diseño de Arquitectura de Colecciones

Diseñe el backend de un módulo para una plataforma de *E-Commerce*. Debe elegir la interfaz y la implementación concreta del Java Collections Framework (java.util) más eficiente para cada una de las siguientes tres funcionalidades:

Requerimiento 1: Sistema de Auditoría de Transacciones Registrar operaciones en el orden exacto en que ocurren. Se requiere insertar miles de registros por segundo al final de la colección y recorrerlos secuencialmente.
Requerimiento 2: Catálogo de Ofertas Relámpago Mantener los productos ordenados automáticamente por precio de menor a mayor. Debe permitir consultar eficientemente el producto más barato y el más caro.
Requerimiento 3: Búsqueda de Sesiones de Usuarios Activos Buscar instantáneamente la sesión de un usuario dado su Token (String único) entre 500,000 usuarios conectados en simultáneo.

Responder y Justificar:

Completar la siguiente tabla comparativa y justificar la decisión con la complejidad algorítmica Big O:

Requerimiento	Interfaz Seleccionada (List, Set, Map, Queue) PDF	Implementación Concreta (ArrayList, TreeSet, HashMap, etc.) PDF	Complejidad Búsqueda	Complejidad Inserción	Justificación Técnica de la Elección
1. Auditoría					
2. Ofertas					
3. Sesiones					

Consigna 3: Optimización Algorítmica ($O(n^2)$ -> $O(n)$)

Un desarrollador escribió el siguiente método para encontrar el primer carácter que no se repite dentro de una cadena de texto (ejemplo: en "barco", el primero es 'b'; en "abarca", el primero es 'r').

```
public static Character primerCaracterNoRepetidoLento(String texto) {
    // Algoritmo ineficiente  $O(n^2)$ 
    for (int i = 0; i < texto.length(); i++) {
        char c = texto.charAt(i);
        boolean duplicado = false;
        for (int j = 0; j < texto.length(); j++) {
            if (i != j && c == texto.charAt(j)) {
                duplicado = true;
                break;
            }
        }
        if (!duplicado) return c;
    }
    return null;
}
```

Responder y Justificar:

- Refactorización a $O(n)$:** Escriba una versión optimizada del método utilizando la colección adecuada de java.util (HashMap, LinkedHashMap, o HashSet) que resuelva el problema realizando como máximo dos recorridos lineales ($O(n)$).
- Análisis de Colisiones:** Si se utiliza un HashMap internamente, explique qué ocurre si dos caracteres distintos generan el mismo hashCode() (Colisión de Hash) y cómo la estructura resuelve este problema sin perder información.

Consigna 4: Caso Teórico de Borde y Colisiones

Considere las siguientes dos instancias de String:

- String s1 = "FB";
- String s2 = "Ea";

Sabiendo que `s1.hashCode() == s2.hashCode()` devuelve true (ambos dan el entero 2236):

1. Si guardamos ambas cadenas en un `HashSet<String>`, ¿cuántos elementos contendrá la colección? Explique cómo utiliza Java los métodos `hashCode()` y `equals()` internamente para determinar si `s2` debe ingresarse o desecharse.
2. ¿Cómo afecta la acumulación de colisiones de hash al rendimiento teórico de un `HashMap` en el peor escenario posible (de $O(1)$ a $O(n)$ o $O(\log n)$)?